

数值计算方法 第一章平时作业

自动化（控制）

1 问题描述

分别以单精度和双精度数据类型用以下近似算法分别计算 π 的近似值：

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots \right) \quad (1)$$

$$\pi = 6 \left(0.5 + \frac{0.5^3}{2 \times 3} + \frac{3 \times 0.5^5}{2 \times 4 \times 5} + \frac{3 \times 5 \times 0.5^7}{2 \times 4 \times 6 \times 7} + \cdots \right) \quad (2)$$

- 假定真值未知，要求结果具有至少 4 位有效数字，给出计算结果；
- 如果采用单精度数据类型要求计算结果达到机器精度，此时结果如何？（测试机器精度：满足 $1 + \varepsilon > 1$ 的最小浮点数）

2 问题分析

本题要求用两种不同的无穷级数公式计算 π 的近似值，并比较单精度与双精度在不同终止条件下的计算精度。

公式一为 **Leibniz 级数**，其通项为 $\frac{(-1)^i}{2i+1}$ ，是一个交替级数，收敛极慢（收敛速度为 $O(1/n)$ ）。公式二是基于 $\arcsin(0.5) = \pi/6$ 的 **快速收敛级数**，其相邻两项之比为

$$\frac{t_{i+1}}{t_i} = \frac{(2i-1)^2}{(2i)(2i+1)} \cdot x^2 \quad (x = 0.5)$$

随 i 增大迅速趋近于 $x^2/4 = 0.0625$ ，收敛快得多。

对于第（1）问，终止条件采用相邻两次近似值的相对误差小于 5×10^{-4} 。对于第（2）问，终止条件改为当新的累积和与旧值完全相等时停止（即 `S_new == S`），这等价于当前项已小于以当前累积和为基准的机器精度，无法再改变浮点数的存储值。

3 程序代码

3.1 第（1）问代码：4 位有效数字

```
fprintf('--- 公式一：Leibniz 级数 ---\n');  
% 单精度
```

```

pi_new_s1 = single(4);
i = 0;
while true
    i = i + 1;
    term = single((-1)^i) / single(2*i + 1);
    pi_old_s1 = pi_new_s1;
    pi_new_s1 = pi_old_s1 + single(4) * term;
    if abs((pi_new_s1 - pi_old_s1) / pi_new_s1) < 5e-4
        break;
    end
end
fprintf('单精度 Leibniz: pi ≈ %.4f, 共迭代 %d 项\n', pi_new_s1, i+1);

% 双精度
pi_new_d1 = 4;
i = 0;
while true
    i = i + 1;
    term = (-1)^i / (2*i + 1);
    pi_old_d1 = pi_new_d1;
    pi_new_d1 = pi_old_d1 + 4 * term;
    if abs((pi_new_d1 - pi_old_d1) / pi_new_d1) < 5e-4
        break;
    end
end
fprintf('双精度 Leibniz: pi ≈ %.4f, 共迭代 %d 项\n', pi_new_d1, i+1);

fprintf('--- 公式二: 快速收敛级数 ---\n');
% 单精度
x = single(0.5); t = x; S = t; i = 1;
while true
    t_new = t * (single(2*i-1)^2) / single(2*i) / single(2*i+1) * x^2;
    S_new = S + t_new;
    if abs((S_new - S) / S_new) < 5e-4
        S = S_new; break;
    end
    t = t_new; S = S_new; i = i + 1;
end
fprintf('单精度 公式二: pi ≈ %.4f, 共迭代 %d 项\n', single(6)*S, i+1);

% 双精度
x = 0.5; t = x; S = t; i = 1;
while true
    t_new = t * ((2*i-1)^2) / (2*i) / (2*i+1) * x^2;
    S_new = S + t_new;

```

```

        if abs((S_new - S) / S_new) < 5e-4
            S = S_new; break;
        end
        t = t_new; S = S_new; i = i + 1;
    end
    fprintf('双精度 公式二: pi ≈ %.4f, 共迭代 %d 项\n', 6*S, i+1);

```

3.2 第（2）问代码：达到机器精度

```

fprintf('--- 公式一: Leibniz 级数（机器精度）---\n');
% 单精度
pi_new_s1 = single(4);
i = 0;
while true
    i = i + 1;
    term = single((-1)^i) / single(2*i + 1);
    pi_old_s1 = pi_new_s1;
    pi_new_s1 = pi_old_s1 + single(4) * term;
    if pi_old_s1 == pi_new_s1 % 新值加不进去，达到机器精度
        break;
    end
end
fprintf('单精度 Leibniz: pi ≈ %.7f, 共迭代 %d 项\n', pi_new_s1, i+1);

fprintf('--- 公式二: 快速收敛级数（机器精度）---\n');
% 单精度
x = single(0.5); t = x; S = t; i = 1;
while true
    t_new = t * (single(2*i-1)^2) / single(2*i) / single(2*i+1) * x^2;
    S_new = S + t_new;
    if S == S_new % 新项加不进去，达到机器精度
        S = S_new; break;
    end
    t = t_new; S = S_new; i = i + 1;
end
fprintf('单精度 公式二: pi ≈ %.7f, 共迭代 %d 项\n', single(6)*S, i+1);

```

4 运行结果

4.1 第（1）问运行结果

```

--- 公式一: Leibniz 级数 ---

```

```

单精度 Leibniz: pi ≈ 3.1424, 共迭代 1275 项
双精度 Leibniz: pi ≈ 3.1424, 共迭代 1275 项
--- 公式二: 快速收敛级数 ---
单精度 公式二: pi ≈ 3.1415, 共迭代 5 项
双精度 公式二: pi ≈ 3.1415, 共迭代 5 项

```

4.2 第（2）问运行结果

```

--- 公式一: Leibniz 级数（机器精度） ---
单精度 Leibniz: pi ≈ 3.1415968, 共迭代 16777217 项
--- 公式二: 快速收敛级数（机器精度） ---
单精度 公式二: pi ≈ 3.1415925, 共迭代 10 项

```

5 结果分析

汇总两问的计算结果如下：

	第（1）问（4 位有效数字）		第（2）问（机器精度，单精度）	
	Leibniz	公式二	Leibniz	公式二
结果	3.1424	3.1415	3.1415968	3.1415925
迭代项数	1275	5	16 777 217	10
准确位数	4 位	4 位	≈5 位	≈7 位

表 1: 两种公式在不同精度要求下的计算结果对比

5.1 Leibniz 级数迭代 1677 万次后为何自动停止

第（2）问中 Leibniz 级数实际迭代了 $16\,777\,217 = 2^{24} + 1$ 次，这个数字不是随机的。单精度浮点数（IEEE 754）的尾数共 23 位，加上隐含的最高位，实际有效位数为 24 位，能精确表示的最大整数为 $2^{24} = 16\,777\,216$ 。当循环变量 i 超过这个值之后，`single(2*i+1)` 的计算结果会因精度不足而被舍入，导致分母不再随 i 增大而变化，新项与上一项之差变为零，终止条件 `pi_old == pi_new` 自然触发。换句话说，循环不是因为“已经足够精确”而停止，而是因为单精度整数表示能力到了极限。这提醒我们，用浮点数做循环计数时，一旦计数值超过精度范围，程序行为可能与预期不符。

5.2 迭代次数越多，结果反而越差

从表 1 可以看出一个反直觉的现象：Leibniz 级数在第（2）问中迭代了 1677 万次，最终结果却只有约 5 位准确数字（3.1415968）；而公式二只迭代 10 次，结果却达到了 7 位（3.1415925），与真值 3.14159265... 更接近。

原因在于**拖尾效应** (Smearing)。Leibniz 级数是交替级数，每一步都在做“加一个正项、减一个负项”的操作，而单精度下每次加减都会引入约 10^{-7} 量级的舍入误差。经过 1677 万次累加，这些误差不断叠加，最终使结果的第 6 位数字就已经不可靠。公式二每项都比前一项小约 $1/16$ ，只需少数几项便能覆盖所需精度，运算次数少，舍入误差自然也少。可以说，Leibniz 是用“勤能补拙”的思路硬算，但在浮点数的世界里，做得越多、错得越多。

5.3 第 (1) 问 Leibniz 结果为 3.1424，而非 3.1415

第 (1) 问要求 4 位有效数字，Leibniz 的输出是 3.1424，与真值 3.14159... 相差约 8×10^{-4} ，第 4 位有效数字就已经不对了。直觉上，终止条件 $|\pi_{\text{new}} - \pi_{\text{old}}| / |\pi_{\text{new}}| < 5 \times 10^{-4}$ 应当保证 4 位有效数字，为什么结果还是偏了？

问题出在终止条件本身。这个条件衡量的是**相邻两次估计值之差**，而 Leibniz 级数是交替级数，每一项从两侧轮流逼近真值——第奇数项偏高，第偶数项偏低。当相邻两项之差恰好小于阈值时，我们停在了某一侧，而不是真值附近。对于收敛慢的级数，“相邻差小”和“离真值近”之间差距明显。公式二收敛快，两者几乎同时满足，所以没有这个问题。这说明对慢收敛级数，用相对误差作终止条件时需要格外小心，或者改用已知真值做误差估计。

5.4 两种公式的收敛速度对比

公式二相邻两项之比为

$$\frac{t_{i+1}}{t_i} = \frac{(2i-1)^2}{(2i)(2i+1)} \cdot x^2 \xrightarrow{i \rightarrow \infty} \frac{x^2}{4} = 0.0625$$

即每项大约是上一项的 $1/16$ ，属于几何级数收敛，5 项之后误差就已低于 10^{-4} 。Leibniz 的第 i 项为 $4/(2i+1)$ ，要使其小于 5×10^{-4} ，需要 $i > 4000$ ，而要小于 10^{-7} （机器精度量级）更需要 $i > 10^7$ ，属于调和级数收敛，极慢。这个差距不只是“快一点慢一点”的问题，而是指数级和线性级的本质区别，在实际数值计算中选择合适的级数展开式至关重要。

6 心得体会

做这道题之前，我以为只要迭代次数够多，结果就会越来越准。但 Leibniz 级数在单精度下跑了 1677 万次，结果反而比公式二跑 10 次更差，这让我第一次真切地感受到舍入误差累积的威力——浮点运算不是“免费的”，每一步都在损耗精度，做得越多不一定越好。另外，终止条件的设计也比我想象的微妙，“相邻差足够小”并不等价于“结果足够准”，对慢收敛的交替级数尤其如此。